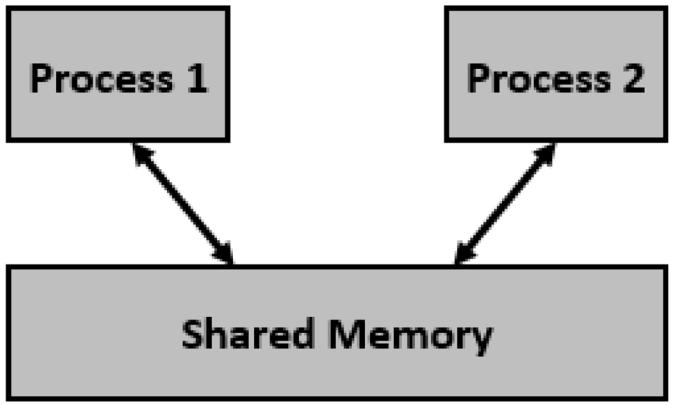
OS – LAB

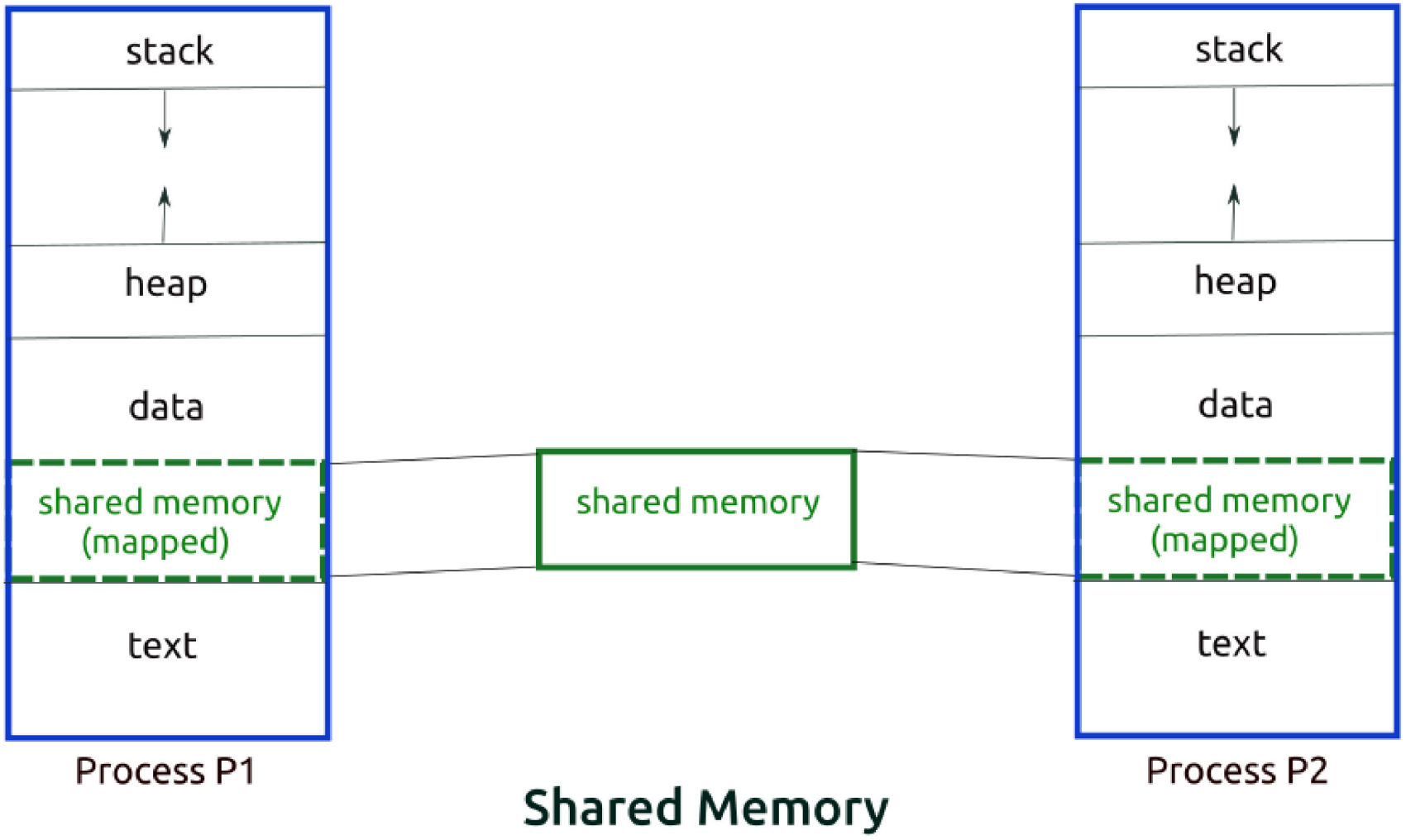
Inter Process Communication (IPC) : Shared Memory

IPC through shared memory

- Shared memory is a memory shared between two or more processes.
- Inter Process Communication through shared memory is a concept where two or more process can access the common memory.
- And communication is done via this shared memory where changes made by one process can be viewed by another process.
- However, why do we need to share memory or some other means of communication?
- Each process has its own address space, if any process wants to communicate some information from its own address space to other processes, then it is only possible with IPC (inter process communication) techniques such as pipes.

- Shared memory is one of the three inter process communication (IPC) mechanisms available under Linux and other Unix-like systems.
- The other two IPC mechanisms are the message queues and semaphores.
- In case of shared memory, a shared memory segment is created by the kernel and mapped to the data segment of the address space of a requesting process.
- A process can use the shared memory just like any other global variable in its address space.




stack
heap
data
shared memory
(mapped)
text
Process P1
shared memory
stack
heap
data
shared memory
(mapped)
text
Process P2
Shared Memory

In the inter process communication mechanisms like the pipes, the work involved in sending data from one process to another is like this:

- Process *P1* makes a system call to send data to Process *P2*.
- The message is copied from the address space of the first process to the kernel space during the system call for sending the message.
- Then, the second process makes a system call to receive the message.
- The message is copied from the kernel space to the address space of the second process.

The shared memory mechanism does away with this copying overhead:

- The first process simply writes data into the shared memory segment.
- As soon as it is written, the data becomes available to the second process.
- Shared memory is the fastest mechanism for inter process communication.

- Shared memory and Semaphores come in two flavors: the traditional System V and the newer POSIX.

- We will discuss System V for shared memory and POSIX for semaphores.

Procedure for using shared memory:

- Find a *key*. Unix uses this key for identifying shared memory segments.
- Use shmget() to allocate a shared memory.
- Use shmat() to attach a shared memory to an address space.
- Use shmdt() to detach a shared memory from an address space.
- Use shmctl() to deallocate a shared memory.

Keys

- To use shared memory, include the following:

```
#include  <sys/types.h>
#include  <sys/ipc.h>
#include  <sys/shm.h>
```

- A key is a value of type key_t. There are three ways to generate a key:
  - Do it yourself – an arbitrary value
  - Use function ftok()
  - Ask the system to provide a private key

Keys

- Do it yourself: use

```
key_t   SomeKey;
SomeKey = 1234;
```

- Use `ftok()` to generate one for you:

```
key_t = ftok(char *path, int ID);
```

  - `path` is a path name (e.g., `"./"`)
  - `ID` is an integer (e.g., `'a'`)
  - Function `ftok()` returns a key of type `key_t`: `SomeKey = ftok("./", 'x');`

- Keys are *global* entities. If other processes know your key, they can access your shared memory.
- Ask the system to provide a private key using `IPC_PRIVATE`.

shmget()

- Allocates/Requests a shared memory segment.

```
int shmget(key_t key, int size, int flags)
```

- The `key` can be either an arbitrary value or one that can be derived from the library function ftok(). The key can also be `IPC_PRIVATE`.
- `size` is the size in bytes of the shared memory segment you want allocated.
- `flags` indicate how you want the segment created and its access permissions. The general rule is just to use `IPCCREAT | 0666`
- `0666` is the usual access permission in Linux in rwx octal format and having the sequence (owner-group-world)

shmget()

- `IPC_CREAT` creates a new segment. Without it, it returns the id for an already existing segment corresponding to the given key.
- This call would return a valid shared memory identifier (used for further calls of shared memory) on success.
- If the return value is `-1,` this means that the request failed / was unsuccessful and no shared memory was allocated.
- The value returned is the `id` of the freshly allocated segment.

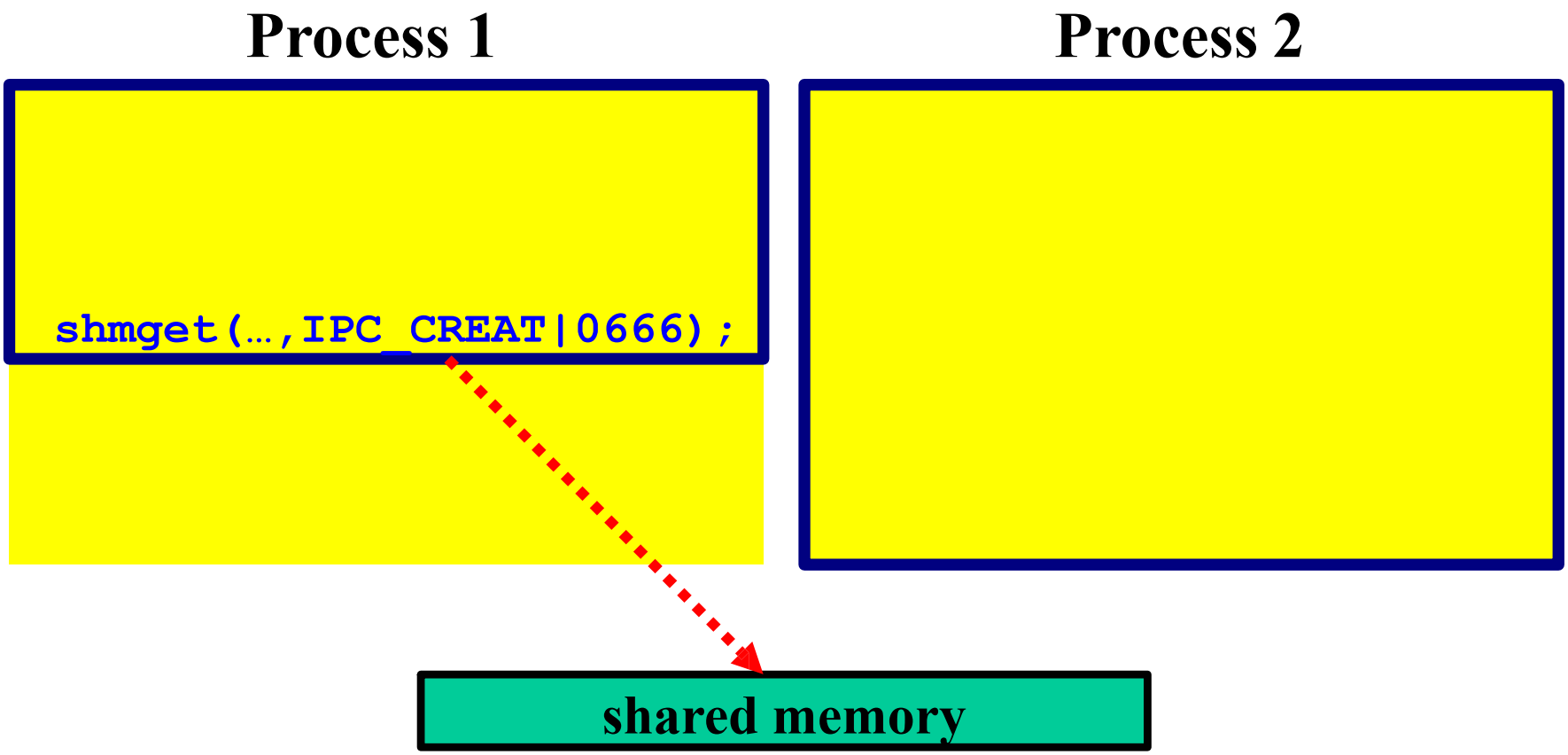
The following creates a shared memory of size `struct Data` with a private key `IPC_PRIVATE`. This is a creation (`IPC_CREAT`) and permits read and write (`0666`).

```
struct Data { int a; double b; char x; };
int ShmID;

ShmID = shmget(
  IPC_PRIVATE, sizeof(struct Data),
  IPC_CREAT | 0666);
```

The following creates a shared memory of size `struct Data` with a an arbitrary key value.

```
struct Data { int a; double b; char x; };
int ShmID;

ShmID = shmget(
  9876 , sizeof(struct Data),
  IPC_CREAT | 0666);
```

After the execution of `shmget()`



Shared memory is allocated; but, is not part of the address space

shmat()

- Use `shmat()` to attach an existing shared memory to an address space:

```
shm_ptr = shmat(
        int     shm_id,     /*  ID  from shmget()       */
        char    *addr,
        int     flags);
```

- `shm_id` is the id as returned by `shmget()` of the shared memory segment you wish to attach.
- `addr` is the address you want the segment mapped at. It's best to pass `NULL` here as the system will choose a suitable unused address to attach the segment at itself.
- `flags` specifies various extra details about how it should be mapped. Just pass in `0` here.

shmat()

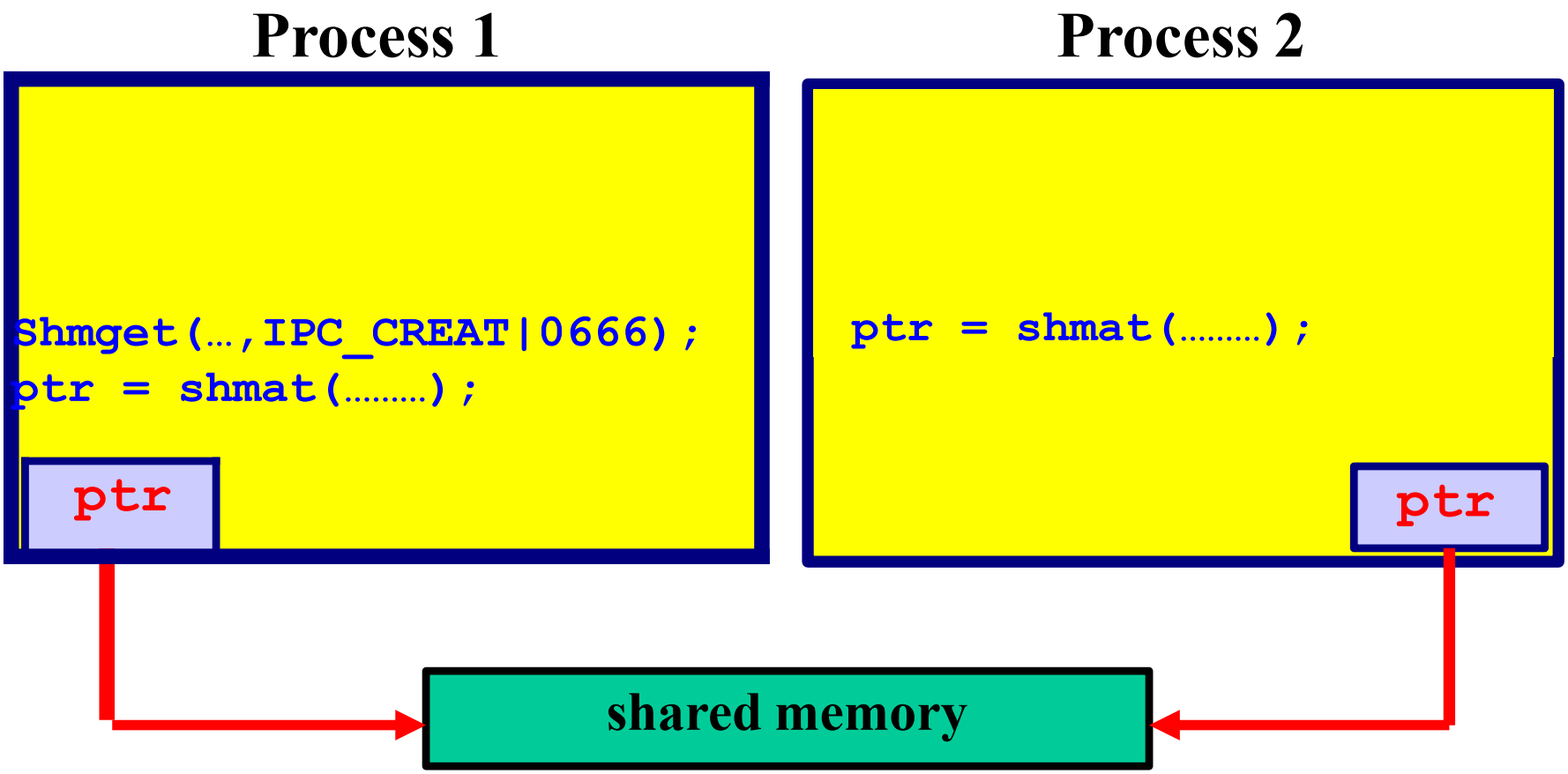
- `shmat()` returns a `void` pointer to the memory.

- This call would return the address of attached shared memory segment on success and `-1` in case of failure.

- What is returned is a pointer to where the segment has been mapped. You can now treat it almost like any other piece of memory you own.

```
struct Data { int a; double b; char x;};
int     ShmID;
key_t   Key;
struct Data *p;

Key = 9876;
ShmID = shmget(Key, sizeof(struct Data),
                    IPC_CREAT | 0666);

p =  (struct Data *) shmat(ShmID, NULL, 0);
if  ((int) p < 0) {
   printf("shmat() failed\n"); exit(1);
}
```

After the execution of `shmat()`



Now processes can access the shared memory

shmdt()

- Detaches the given segment.
- You need to do this when you no longer require the shared memory segment any longer, e.g. when you program is shutting down

```
int shmdt(const void* addr);
```

- `addr` is the address where the shared memory segment you wish to detach is mapped.
- The to-be-detached segment must be the address returned by the `shmat()` system call.

- This call would return `0` on success and `-1` in case of failure.

shmctl()

- A system call that performs control operation for a System V shared memory segment.

```
int shmctl(int id, int cmd, struct shmid_ds* buf);
```

- `id` is the segment's id as returned by `shmget()`.
- `cmd` is the command you want to perform. Most used are: `IPCSTAT`, `IPCSET`, and `IPCRMID`
- `buf` points to a buffer used by `IPCSTAT` and `IPCSET`. When `cmd` is set to `IPCRMID`, this should be `NULL`.

shmctl()

IPCSTAT & IPCSET

- These are for getting and setting information about the shared memory segment.
- For more information , type `man shmctl`.

IPCRMID

- Marks the segment to be destroyed.
- The segment is destroyed only after the last process has detached it.

Detaching vs removing shared memory

- To detach a shared memory, use

```
shmdt(shm_ptr);
```

`shm_ptr` is the pointer returned by `shmat()`.

- After a shared memory is detached, it is still there. You can re-attach and use it again.

- To remove a shared memory, use

```
shmctl(shm_ID, IPC_RMID, NULL);
```

`shm_ID` is the shared memory ID returned by `shmget()`.

- After a shared memory is removed, it no longer exists.